

Zero to production.

From an empty file to a production backend, and from using the framework to understanding it — in five parts. Every snippet is self-contained and self-asserting:

`tests/test_guide.py` in the repo executes all of them on every push, so this page cannot drift. Canonical source: docs/GUIDE.md.

Part I

Basics

Install the package, serve the first route, and learn the request path: routing, bodies, validation, dependencies.

CHAPTER 01

Install

One package, zero required dependencies. The core runs on stdlib alone; servers, drivers, and test tools arrive as extras. This snippet asserts the installed package, so CI proves the floor this guide stands on.

Install it:

Notes: `pip install -e ".[dev]"` from a checkout gets everything at once. Requires Python 3.10 or newer; CI covers 3.10 through 3.13.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from importlib.metadata import version

v = version("ikarem")
major = int(v.split(".")[0])
assert major >= 1, v
```

CHAPTER 02

First light

Create an app, return a dict. Dicts, lists, strings, and bytes become responses automatically — ceremony is reserved for the cases that need it.

Serve it for real with any ASGI server (`app.run()` needs `pip install ikarem[server]`), or keep driving it in-process with `TestClient` — every chapter below uses the client, and your test suite should too. Notes: `debug=True` adds tracebacks to 500s (never in production); `enable_docs=False` hides `/openapi.json` and `/docs` but never the health probes.

Runnable — copy into a file and run with `python` . Executed in CI by `tests/test_guide.py` .

```
from ikarem import Ikarem
from ikarem.testing import TestClient

app = Ikarem()

@app.get("/")
async def home(req):
    return {"hello": "ikarem"}

c = TestClient(app)
r = c.get("/")
assert r.status_code == 200 and r.json() == {"hello": "ikarem"}
assert c.get("/missing").status_code == 404
```

CHAPTER 03

Routing with intent

Routes declare converters (`{uid:int}`), plus `float` , `uuid` , `path`), so bad segments 404 instead of crashing handlers. A path that matches nothing is 404; a path that matches with the wrong verb is 405. Name every route for free with the handler name, and reverse it with `url_for` .

Notes: converters compile to regex once at registration; static routes resolve in $O(1)$. Unknown converters fail at startup with the valid list, not on first traffic. Close misses get "Did you mean" 404s.

Runnable — copy into a file and run with `python` . Executed in CI by `tests/test_guide.py` .

```
from ikarem import Ikarem
from ikarem.testing import TestClient

app = Ikarem(enable_docs=False)

@app.get("/users/{uid:int}")
async def get_user(req, uid: int):
    return {"uid": uid}

c = TestClient(app)
assert c.get("/users/7").json() == {"uid": 7}
assert c.get("/users/abc").status_code == 404
assert c.post("/users/7", body={}).status_code == 405
assert app.router.url_for("get_user", uid=7) == "/users/7"
```

CHAPTER 04 —

Bodies with boundaries

Read bodies explicitly (`await req.json()` , `await req.form()` , `await req.body(max_bytes=...)`) so oversized payloads become 413s at a limit you chose, not memory pressure you didn't. The app-wide `max_body_bytes=` sets the floor; per-call `max_bytes=` overrides it.

Notes: `Request.json()` caps at 10MB by default. Streaming uploads should always pass an explicit cap. 413 responses still travel the middleware pipeline, so observability headers survive them.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from ikarem import Ikarem
from ikarem.testing import TestClient

app = Ikarem(enable_docs=False)

@app.post("/echo")
async def echo(req):
    return await req.json()

@app.post("/small")
async def small(req):
    return {"n": len(await req.body(max_bytes=10))}

c = TestClient(app)
assert c.post("/echo", body={"a": 1}).json() == {"a": 1}
assert c.post("/small", body="12345").json() == {"n": 5}
assert c.post("/small", body="x" * 100).status_code == 413
```

CHAPTER 05

Validation that reads like the docs

`Schema` models coerce and check; `Field()` states bounds once and they surface in errors, OpenAPI, and MCP schemas together. `extra="forbid"` rejects unknown keys — the default ignores them, which is how mass assignment sneaks in. Body-only handlers (no `req`) are allowed wherever the request itself isn't needed.

Notes: nested models validate recursively; coerced values (`"2"` to `2`) are what the handler receives. `json_schema()` powers OpenAPI and MCP input schemas from the same definition.

Runnable — copy into a file and run with `python` . Executed in CI by

`tests/test_guide.py` .

```
from ikarem import Field, Ikarem, Schema
from ikarem.testing import TestClient

class Item(Schema):
    name: str = Field(..., min_length=1, max_length=80)
    qty: int = Field(1, ge=1, le=99)

class Order(Schema, extra="forbid"):
    item: Item
    express: bool = False

app = Ikarem(enable_docs=False)

@app.post("/orders")
async def create(order: Order):
    return {"name": order.item.name, "qty": order.item.qty, "express": order.express}

c = TestClient(app)
good = {"item": {"name": "apple", "qty": "2"}, "express": True}
assert c.post("/orders", body=good).json() == {"name": "apple", "qty": 2, "express": True}
assert c.post("/orders", body={"item": {"qty": 1}}).status_code == 400
assert c.post("/orders", body={"item": {"name": "x"}, "hack": 1}).status_code == 400
```

CHAPTER 06

Dependencies without magic

`Depends()` declares inputs; the framework builds them per request, caches repeats, and runs yield-dependency finalizers after the response is sent — even on the exception path. Dependencies are plain callables: test them without the framework.

Notes: nesting works to any depth; cycles fail at startup with the path.

`Depends(fn, use_cache=False)` opts out of the per-request cache. Bare

`Depends()` with no callable is a startup error naming the parameter.

Runnable — copy into a file and run with `python`. Executed in CI by

`tests/test_guide.py`.

```
from ikarem import Depends, Ikarem
from ikarem.testing import TestClient

closed = []

def settings():
    return {"currency": "USD"}

async def ledger_conn(req):
    conn = {"open": True}
    yield conn
    closed.append("closed")

app = Ikarem(enable_docs=False)

@app.get("/balance")
async def balance(req, cfg=Depends(settings), db=Depends(ledger_conn)):
    return {"currency": cfg["currency"], "open": db["open"]}

c = TestClient(app)
assert c.get("/balance").json() == {"currency": "USD", "open": True}
assert closed == ["closed"]
```

Security

Authentication, sessions, middleware order, and errors. This part is deliberately strict: people copy guide code, so every snippet here is written the way production should look.

CHAPTER 07

Auth that says no clearly

Stateless JWT (HS256, stdlib only) plus role and scope guards. 401 means anonymous, 403 means authenticated but not allowed — clients can act on the difference. Secrets come from config and are refused at startup when they are still defaults.

Notes: production sets `IKAREM_AUTH_SECRET` (the strict env-or-raise shape is chapter 8's); tokens always carry an explicit `expires_in` — readers copy this call, so the default here is 15 minutes, not forever. `verify_token` rejects algorithm confusion (`alg=none`), expired tokens, and missing `sub` . Service-to-service keys use `APIKeyAuth` with static keys or an async `lookup=` . Passwords hash with `pbkdf2` — see the next chapter for the full login shape.

Runnable — copy into a file and run with `python` . Executed in CI by `tests/test_guide.py` .

```
import os
import time

from ikarem import Depends, Ikarem, create_token, require_roles, require_scopes, verify_token
from ikarem.testing import TestClient

SECRET = os.environ.get("IKAREM_AUTH_SECRET") or "guide-dev-only-secret"
app = Ikarem(enable_docs=False, auth_secret=SECRET)

@app.get("/admin")
async def admin(req, claims=Depends(require_roles("admin"))):
    return {"sub": claims["sub"]}

@app.get("/files")
async def files(req, claims=Depends(require_scopes("read"))):
    return {"n": 2}

c = TestClient(app)
assert c.get("/admin").status_code == 401
user = create_token("u2", SECRET, roles=["user"])
assert c.get("/admin", headers={"authorization": f"Bearer {user}"}).status_code == 403
root = create_token("u1", SECRET, expires_in=900, roles=["admin"], scope="read")
claims = verify_token(root, SECRET)
assert 0 < claims["exp"] - time.time() <= 900 # explicit 15-minute life
assert c.get("/admin", headers={"authorization": f"Bearer {root}"}).json() == {"sub": "u1"}
assert c.get("/files", headers={"authorization": f"Bearer {root}"}).json() == {"n": 2}
```

CHAPTER 08

Logins without shortcuts

Passwords hash with pbkdf2 and verify in constant time — never `==` against plaintext, never stored plaintext. The session secret loads from the environment (no hardcoded fallback that ships to production), and a successful login starts a fresh session so a pre-login cookie can't be fixed onto a victim.

Notes: a random per-process fallback would sign cookies with a different key in every uvicorn worker (chapter 21) — users logged out at random, and nothing

would warn you because random is not a default. So the fallback here requires the explicit `IKAREM_DEV=1` flag, and anything else raises with the remedy. Unknown usernames verify against a dummy hash, so timing reveals nothing. Cookies arrive `HttpOnly` with `SameSite=Lax` by default; `Secure` is opt-in via `SessionMiddleware(secure=True)` — turn it on the moment you serve HTTPS. The `TestClient` cookie jar persists login across requests, so flows test exactly as browsers behave.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import os

from ikarem import (
    CSRFMiddleware,
    Ikarem,
    RateLimitMiddleware,
    SessionMiddleware,
    Unauthorized,
    check_password,
    csrf_token,
    hash_password,
)

from ikarem.testing import TestClient

def resolve_secret():
    secret = os.environ.get("IKAREM_SESSION_SECRET")
    if secret:
        return secret
    if os.environ.get("IKAREM_DEV") == "1":
        return "dev-only-insecure-secret" # throwaway local runs
    raise RuntimeError("set IKAREM_SESSION_SECRET to a long random value (or IKAREM_DEV=1 locally)")

os.environ.setdefault("IKAREM_DEV", "1") # delete this line in production
SECRET = resolve_secret()

# the guard, proven both ways: the production shape refuses to boot
_saved = (os.environ.pop("IKAREM_DEV", None), os.environ.pop("IKAREM_SESSION_SECRET", None))
try:
    try:
        resolve_secret()
        assert False, "must refuse without secret or dev flag"
    except RuntimeError as e:
        assert "IKAREM_SESSION_SECRET" in str(e)
finally:
    if _saved[0] is not None:
        os.environ["IKAREM_DEV"] = _saved[0]
    if _saved[1] is not None:
        os.environ["IKAREM_SESSION_SECRET"] = _saved[1]

DUMMY_HASH = hash_password("no-such-user") # unknown users verify here: same timing
USERS = {"amy": hash_password("s3cret")} # seeded hash, never the password
```

```
app = Ikarem(enable_docs=False, session_secret=SECRET)
app.use(RateLimitMiddleware(per_minute=11)) # logins are brute-forced: budget them
app.use(SessionMiddleware())
app.use(CSRFMiddleware())

@app.get("/csrf")
async def csrf(req):
    return {"t": csrf_token(req)}

@app.post("/login")
async def login(req):
    form = await req.form()
    pw_hash = USERS.get(form.get("user", "")) or DUMMY_HASH
    if not check_password(form.get("pw", ""), pw_hash):
        raise Unauthorized("bad credentials")
    req.session.clear() # fresh session on login: fixation-safe
    req.session["uid"] = "u1"
    return {"ok": True}

@app.post("/logout")
async def logout(req):
    req.session.clear()
    return {"ok": True}

@app.get("/me")
async def me(req):
    uid = req.session.get("uid")
    if not uid:
        raise Unauthorized("anonymous")
    return {"uid": uid}

CT = "application/x-www-form-urlencoded"
c = TestClient(app)
assert c.get("/me").status_code == 401

def token():
    return c.get("/csrf").json()["t"]
```

```

assert (
    c.post("/login", body="user=amy&pw=wrong", content_type=CT, headers={"x-csrf-token":
token()}).status_code
    == 401
)
assert (
    c.post("/login", body="user=noone&pw=x", content_type=CT, headers={"x-csrf-token": token()}).status_code
    == 401
)
good = c.post("/login", body="user=amy&pw=s3cret", content_type=CT, headers={"x-csrf-token": token()})
assert good.json() == {"ok": True}
sc = "; ".join(v for k, v in good.headers_list if k.lower() == "set-cookie")
assert "httponly" in sc.lower() and "samesite=lax" in sc.lower() and "secure" not in sc.lower()
assert c.get("/me").json() == {"uid": "u1"}
assert c.post("/logout", headers={"x-csrf-token": token()}).status_code == 200
assert c.get("/me").status_code == 401
assert c.get("/me").status_code == 429 # budget of 11 spent: brute force pays here

```

CHAPTER 09

Middleware on purpose

The onion: each layer sees the request going in and the response coming out, or short-circuits. Order is the feature — request IDs first, gates early, headers last. A `validate_config(app)` hook lets middleware fail at boot with the remedy.

Notes: raising `Unauthorized` / `Forbidden` inside middleware short-circuits with the same rendering as handler errors. Sessions must precede CSRF — reversed order refuses to boot. Keep middleware free of business logic: gates and headers, nothing else.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from ikarem import Ikarem, RequestIDMiddleware, SecurityHeadersMiddleware
from ikarem.testing import TestClient

order = []

app = Ikarem(enable_docs=False)
app.use(RequestIDMiddleware())
app.use(SecurityHeadersMiddleware())

async def audit(req, call_next):
    order.append("in")
    resp = await call_next(req)
    order.append("out")
    resp.headers["x-audited"] = "yes"
    return resp

app.use(audit)

@app.get("/")
async def home(req):
    return {"ok": True}

c = TestClient(app)
r = c.get("/")
assert r.headers["x-request-id"] and r.headers["x-frame-options"] == "DENY"
assert r.headers["x-audited"] == "yes" and order == ["in", "out"]
```

CHAPTER 10

Errors with remedies

`abort(status, detail)` fails tersely through the pipeline so error responses keep request IDs and security headers. Custom handlers map exception types, most-specific first. Framework law: every error names the fix.

Notes: unhandled exceptions are 500s with no leak (tracebacks only under `debug=True`). 404s suggest close matches. Handler errors render inside the middleware pipeline — headers included.

Runnable — copy into a file and run with `python` . Executed in CI by

`tests/test_guide.py` .

```
from ikarem import Ikarem, JSONResponse, abort
from ikarem.testing import TestClient

app = Ikarem(enable_docs=False)

@app.exception_handler(ValueError)
async def _bad(req, exc):
    return JSONResponse({"detail": f"bad value: {exc}"}, status_code=422)

@app.get("/widget/{wid:int}")
async def widget(req, wid: int):
    if wid == 0:
        abort(404, "no such widget")
    return {"wid": wid}

@app.get("/parse")
async def parse(req):
    raise ValueError("qty")

c = TestClient(app)
assert c.get("/widget/0").status_code == 404
assert c.get("/parse").status_code == 422
assert c.get("/widget/3").json() == {"wid": 3}
```

Part III —————

Data and work

Persistence, background work, time, and the blocking-code trap. The throughline: the framework never hides durability semantics — fire-and-forget, at-least-once, and transactional each look different.

Configuration without surprises

Layered and explicit: defaults, then kwargs, then dict, then `IKAREM_*` environment variables. Typed reads with `cast=`. No settings module, no import-time environment sniffing.

Notes: secrets (`auth_secret` , `session_secret`) also read from `IKAREM_AUTH_SECRET` / `IKAREM_SESSION_SECRET` . Startup validation refuses to serve authenticated routes on default secrets.

Runnable — copy into a file and run with `python` . Executed in CI by `tests/test_guide.py` .

```
from ikarem import Ikarem

app = Ikarem(enable_docs=False, debug=False, page_size=20)

assert app.config.get("page_size") == 20
assert app.config.get("missing", "fallback") == "fallback"
assert app.config.get("page_size", cast=str) == "20"
assert app.config.get("debug") is False
```

Data that survives

One connector interface, four engines. Placeholders are always `?` ; rows are plain dicts; drivers import lazily with the exact extra named on failure. Transactions commit on clean exit and roll back on error.

Notes: SQLite serializes on a worker-side lock (single-lane by design, WAL on); Postgres/MySQL pools rebuild per event loop. Point `IKAREM_DB_URL` at Postgres when writes outgrow one lane — the suite is green on both.

Runnable — copy into a file and run with `python` . Executed in CI by `tests/test_guide.py` .

```
import asyncio

from ikarem import Ikarem
from ikarem.db import DatabasePlugin

app = Ikarem(enable_docs=False)
app.register(DatabasePlugin("sqlite:///memory:"))

async def main():
    await app.startup()
    try:
        db = app.state_db
        await db.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, text TEXT)")
        async with db.transaction():
            await db.execute("INSERT INTO notes (text) VALUES (?)", "buy milk")
        rows = await db.fetch_all("SELECT * FROM notes")
        assert [r["text"] for r in rows] == ["buy milk"]
    finally:
        await app.shutdown()

asyncio.run(main())
```

CHAPTER 13

Work that outlives the deploy

`BackgroundTasks` fire after the response and vanish on restart. The durable queue survives it: portable leases, exponential-backoff retries, parked dead jobs instead of silent loss. Drain with `ikarem worker`.

Notes: unknown task names and bad payloads fail the job, not the worker.

`depth()` exposes queue length for `/readyz`-style checks and dashboards.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import asyncio

from ikarem import BackgroundTasks, Ikarem
from ikarem.db import DatabasePlugin
from ikarem.queue import QueuePlugin, task
from ikarem.testing import TestClient

fired, done = [], []

@task("guide-receipt")
async def receipt(order_id: int):
    done.append(order_id)

app = Ikarem(enable_docs=False)
app.register(DatabasePlugin("sqlite:///memory:"))
app.register(QueuePlugin())

@app.post("/orders")
async def order(req, bg: BackgroundTasks):
    bg.add(fired.append, "ack")
    await req.app.state_queue.enqueue("guide-receipt", {"order_id": 7})
    return {"ok": True}

async def main():
    await app.startup()
    try:
        await app.state_queue.enqueue("guide-receipt", {"order_id": 7})
        assert await app.state_queue.run_one() is True
        assert done == [7]
    finally:
        await app.shutdown()

asyncio.run(main())
assert TestClient(app).post("/orders", body={}).json() == {"ok": True}
assert fired == ["ack"]
```

Time, kept by the app

Cron plus intervals with an explicit start — nothing runs unless started. All timing flows through `tick(now)`, so tests travel through time instead of sleeping. In production, `SchedulerPlugin` ties the loop to lifespan; `app.scheduler()` is the public accessor either way.

Notes: overlapping runs of one job never stack — a still-running job is skipped that tick. Failures are counted and logged with tracebacks, never swallowed.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import asyncio
import time

from ikarem import Ikarem
from ikarem.scheduler import SchedulerPlugin

app = Ikarem(enable_docs=False)
ran = []

@app.every(60)
async def heartbeat():
    ran.append(1)

@app.cron("@daily")
async def rollup():
    ran.append("daily")

async def main():
    sched = app.scheduler()
    assert [j.name for j in sched.jobs] == ["heartbeat", "rollup"]
    await sched.tick(now=time.time() + 61)
    assert ran == [1]
    app.register(SchedulerPlugin(poll=0.01))
    await app.startup()
    assert getattr(app, "state_scheduler", None) is sched
    await app.shutdown()

asyncio.run(main())
```

CHAPTER 15

Blocking code belongs in threads

One event loop serves every request on a worker. A blocking call — `time.sleep`, a sync driver, DNS — stalls all of them, and the stall is invisible in profiles of your code because the loop is simply absent. Push blocking work to threads; keep

`async` handlers non-blocking. The unambiguous rule, proven by the thread IDs below: IKAREM calls `async` handlers on the event-loop thread, and `def` handlers on that same thread — there is no hidden threadpool. A `def` handler is fine if it returns fast; a blocking call anywhere on the loop stalls every request on that worker.

Notes: this is the most common Flask/Django carryover bug — sync ORM calls pasted into `async` handlers. Convert the driver first (asyncpg, aiomysql), thread the rest with `asyncio.to_thread` (default pool: at most 32 threads, fewer on small machines). Never `asyncio.Lock` around threaded work; SQLite already serializes on a worker-side threading lock.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import asyncio
import threading
import time

from ikarem import Ikarem
from ikarem.testing import TestClient

def slow_hash(n):
    time.sleep(0.01) # blocking stand-in: hashing, sync drivers, DNS
    return n * 2

app = Ikarem(enable_docs=False)

@app.get("/loop")
async def loop_h(req):
    return {"t": threading.get_ident()}

@app.get("/sync")
def sync_h(req):
    return {"t": threading.get_ident()}

@app.get("/hash/{n:int}")
async def hash_h(req, n: int):
    return {"r": await asyncio.to_thread(slow_hash, n)}

@app.get("/thread")
async def thread_h(req):
    return {"t": await asyncio.to_thread(threading.get_ident)}

c = TestClient(app)
t_loop = c.get("/loop").json()["t"]
assert c.get("/sync").json()["t"] == t_loop # same thread: blocking stalls all
assert c.get("/thread").json()["t"] != t_loop # to_thread escapes the loop
assert c.get("/hash/21").json() == {"r": 42}
```

Structure and shipping

Realtime, files, organization, testing, lifespan, deployment. The part where a project stops being an app and starts being a system.

Talking back live

WebSocket routes plus in-process rooms: join on connect, broadcast to everyone but the sender, leave in a `finally`. Disconnects raise the specific `WebSocketDisconnect` — catch that, not bare `RuntimeError`, so real bugs still surface. The snippet proves both halves: route wiring through a real connection, and the broadcast itself with two members.

Notes: rooms are single-process by design (chapter 21 covers what that implies under workers); an external broker slots behind the same join/broadcast shape. `receive_text` skips handshake frames. Two live scripted clients can't overlap deterministically through one `ws_connect` driver, so the member half is proven against the same `Room` object the route uses — no mocks of the broadcast itself.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import asyncio

from ikarem import Ikarem, Room, WebSocketDisconnect
from ikarem.testing import TestClient

room = Room()
app = Ikarem(enable_docs=False)

@app.websocket("/chat")
async def chat(ws):
    await ws.accept()
    await room.join(ws)
    try:
        while True:
            await room.broadcast(await ws.receive_text(), exclude=ws)
    except WebSocketDisconnect:
        pass
    finally:
        room.leave(ws)

class FakeWS:
    def __init__(self):
        self.sent = []

    async def send_text(self, text):
        self.sent.append(text)

async def main():
    spy = FakeWS()
    await room.join(spy) # a live member before the scripted client arrives
    sent = await TestClient(app).ws_connect(
        "/chat", incoming=[{"type": "websocket.connect"}, {"text": "hi-there"}]
    )
    assert "websocket.accept" in [m["type"] for m in sent]
    # the /chat route really broadcast through the room: the spy got it,
    # and the sender was excluded (nothing else was in the room to echo)
    assert spy.sent == ["hi-there"]
    room.leave(spy)
```

```
asyncio.run(main())  
assert len(room) == 0
```

CHAPTER 17

Files in, files out

Static directories serve with symlink-escape and prefix-collision guards — both were real bugs, so both are asserted here, not just `..`. `FileResponse` streams downloads with `Content-Disposition` surviving middleware. Uploads arrive parsed as `UploadFile` with size caps.

Notes: mount helpers fail loudly on missing directories. The `realpath` guard compares both sides, so neither `..` nor symlinks escape the root. The symlink branch skips where the OS refuses symlinks (some Windows runners) — the enforcement that always runs is

```
tests/test_hardening.py::test_static_blocks_symlink_escape
```

 on Linux CI.

Runnable — copy into a file and run with `python`. Executed in CI by

```
tests/test_guide.py
```

.

```
import os
import tempfile
from pathlib import Path

from ikarem import FileResponse, Ikarem
from ikarem.testing import TestClient

base = Path(tempfile.mkdtemp(prefix="guide-static-"))
pub = base / "pub"
pub.mkdir()
(pub / "ok.txt").write_text("hello file")
sibling = base / "pub-evil" # shared prefix, different directory
sibling.mkdir()
(sibling / "secret.txt").write_text("nope")
outside = base / "outside.txt"
outside.write_text("nope")
try:
    os.symlink(outside, pub / "link.txt")
    link_ok = True
except OSError:
    link_ok = False # symlinks need privileges on some machines

app = Ikarem(enable_docs=False)
app.mount_static("/static", str(pub))

@app.get("/receipt")
async def receipt(req):
    return FileResponse(str(pub / "ok.txt"), filename="receipt.txt")

c = TestClient(app)
assert c.get("/static/ok.txt").text == "hello file"
assert c.get("/static/../pub-evil/secret.txt").status_code == 404
if link_ok:
    assert c.get("/static/link.txt").status_code == 404
dl = c.get("/receipt")
assert dl.status_code == 200 and "attachment" in dl.headers["content-disposition"]
```

Structure that scales

`Blueprint` s group routes with their own hooks and error handlers; `MethodView` puts one resource's verbs in one class with full DI per method. Both compile to the same plans as plain routes — zero per-request overhead for the organization.

Notes: blueprint routes are named `blueprint.handler`, so `url_for` stays unambiguous across groups. Blueprint error handlers scope to their routes; app handlers catch the rest.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from ikarem import Blueprint, Ikarem, MethodView, Schema
from ikarem.testing import TestClient

class ItemIn(Schema):
    name: str

class Items(MethodView):
    async def get(self, req):
        return {"items": []}

    async def post(self, req, item: ItemIn):
        return {"name": item.name}, 201

api = Blueprint("api", url_prefix="/api")
app = Ikarem(enable_docs=False)

@api.get("/ping")
async def ping(req):
    return {"pong": True}

app.register_blueprint(api)
app.route("/items", Items.methods())(Items.as_view("items"))

c = TestClient(app)
assert c.get("/api/ping").json() == {"pong": True}
assert c.get("/items").json() == {"items": []}
assert c.post("/items", body={"name": "a"}).status_code == 201
assert c.post("/items", body={}).status_code == 400
assert app.router.url_for("api.ping") == "/api/ping"
```

Tests that behave like browsers

`TestClient` keeps cookies, speaks every verb, and runs startup — login flows test exactly as browsers behave. `app.check()` audits handlers statically: cycles, bare `Depends()`, duplicate routes, auth gaps.

Notes: one client per thread — loop-bound resources (pools) survive across requests, matching production. Gate deploys on `check` in CI.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from ikarem import Ikarem
from ikarem.testing import TestClient

app = Ikarem(enable_docs=False)

@app.get("/users")
async def users(req):
    return {"n": 1}

report = app.check()
assert report["errors"] == []
assert any(r["path"] == "/users" for r in report["routes"])
c = TestClient(app)
assert c.get("/users").json() == {"n": 1}
miss = c.get("/user")
assert miss.status_code == 404 and "Did you mean" in miss.text
```

CHAPTER 20

Lifespan, wired once

`on_startup` / `on_shutdown` order your boot: tables, queues, schedulers. Plugins hook the same lifecycle with dependency sorting. ASGI lifespan messages drive it all under a real server.

Notes: startup compiles every handler plan, validates config (secrets, middleware order), then runs hooks and plugin startups. Shutdown reverses plugins before app hooks.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import asyncio

from ikarem import Ikarem

events = []
app = Ikarem(enable_docs=False)

@app.on_startup
async def up():
    events.append("up")

@app.on_shutdown
async def down():
    events.append("down")

async def main():
    await app.startup()
    await app.startup() # idempotent: second call is a no-op
    await app.shutdown()

asyncio.run(main())
assert events == ["up", "down"]
```

CHAPTER 21

Deployment without drama

One process per worker, so single-process state (rooms, memory caches, rate-limit buckets) multiplies per worker. Share what must be shared (`RedisCache` behind the same interface), pin sticky routing or accept room locality, configure from the

environment, and gate the deploy on `check`. The full production shape — Dockerfile, compose with Postgres, env table — lives in `docs/DEPLOY.md`.

Serve it: `uvicorn myapp:app --workers 4 ( pip install ikarem[server] )`. One worker is one room, one memory cache, one rate-limit table — size `ConcurrencyLimitMiddleware` and idempotency TTLs for that reality, or share them through Redis. Every worker also needs the same `IKAREM_SESSION_SECRET`: chapter 8's guard refuses to boot without one, because a per-process secret would log users out at random. Health: `/healthz` for liveness, `/readyz` for readiness (503 while the DB is down), `/metrics` for latency and queue depth.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import os

from ikarem import Ikarem

os.environ["IKAREM_PAGE_SIZE"] = "25"
try:
    app = Ikarem(enable_docs=False, page_size=20)
    assert app.config.get("page_size") == 25 # env wins over kwargs
finally:
    del os.environ["IKAREM_PAGE_SIZE"]

assert app.check()["errors"] == []
```

Part V —————

Mastery

Docs, plans, agents, maintenance. Using the framework ends here; understanding it starts here.

CHAPTER 22 —————

Docs and tools for free

OpenAPI, the route manifest, and MCP tools all derive from the same compiled plans — query shapes, bodies, and auth boundaries cannot drift between your docs, your agents, and your runtime.

Notes: `ikarem inspect` prints the manifest for LLM context; `site/llms.txt` is the one-page framework manual. Serve routes as tools with `ikarem mcp myapp:app`.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import asyncio
import os

from ikarem import Depends, Ikarem, Schema, require_roles
from ikarem.compiled import describe_app
from ikarem.openapi import build_openapi

SECRET = os.environ.get("IKAREM_AUTH_SECRET") or "guide-dev-only-secret"

class Item(Schema):
    name: str

app = Ikarem(enable_docs=False, auth_secret=SECRET)

@app.post("/items")
async def create_item(item: Item):
    """Create an item."""
    return {"name": item.name}

@app.get("/admin")
async def admin(req, claims=Depends(require_roles("admin"))):
    return {"sub": claims["sub"]}

spec = build_openapi(app)
assert spec["paths"]["/items"]["post"]["summary"] == "Create an item."
assert spec["paths"]["/admin"]["get"]["security"] == [{"bearerAuth": []}]
manifest = describe_app(app)
assert manifest["count"] == 2
tools = {t["name"]: t for t in app.mcp_tools()}
assert set(tools) == {"create_item", "admin"}
out = asyncio.run(app.mcp_call("create_item", {"name": "apple"}))
assert out["isError"] is False
```

Under the hood: plans, not reflection

Signatures parse once into handler plans; per-request resolution is dict lookups.

`get_plan` caches by handler identity — the identity check below proves compilation happens once, whatever the request volume. That is the core of the performance story; the speed itself is proven by `bench/bench_switch.py` (same harness, same process) and `bench/load.py` (~75k sustained requests over real uvicorn, zero 5xx).

Notes: touch the hot path and the bench decides, not adjectives. No per-request `inspect.signature`, no regex where a dict works — compile once, then dict lookups. Read `compiled.py` to verify, `bench/` to measure.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from ikarem import Ikarem
from ikarem.compiled import get_plan
from ikarem.testing import TestClient

app = Ikarem(enable_docs=False)

@app.get("/u/{uid:int}")
async def u(req, uid: int, limit: int = 5):
    return {"uid": uid, "limit": limit}

assert get_plan(u) is get_plan(u)
c = TestClient(app)
assert c.get("/u/3", query="limit=2").json() == {"uid": 3, "limit": 2}
```

CHAPTER 24

Agents are users too

LLM clients consume the same contracts: manifest for planning, MCP tools for calling, `llms.txt` for the manual, `AGENTS.md` for contributor laws. Build agent-facing features by describing routes, not by special cases.

Notes: auth boundaries propagate into tool descriptions, so agents see

`Requires Authorization` before they call. Keep docstrings to one true first line — it becomes the tool summary.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
from ikarem import Ikarem
from ikarem.compiled import describe_app, describe_route

app = Ikarem(enable_docs=False)

@app.get("/orders/{oid:int}")
async def order(req, oid: int, verbose: bool = False):
    """Fetch one order."""
    return {"oid": oid}

(route,) = [r for r in app.router.routes if r.path == "/orders/{oid:int}"]
desc = describe_route(route)
assert desc.path_params == ["oid"] and desc.doc.startswith("Fetch one")
entry = describe_app(app)["routes"][0]
assert entry["handler"] == "order" and entry["summary"] == "Fetch one order."
```

CHAPTER 25

Staying honest in production

The framework stays fast and honest the same way your app does: every behavior ships with a test, errors name remedies, deprecations warn with versions and replacements, and the changelog is cut from commits. Your next moves: claim a row in `docs/ECOSYSTEM.md`, steal a recipe from `docs/COOKBOOK.md`, and read `CONTRIBUTING.md` before the first PR.

Runnable — copy into a file and run with `python`. Executed in CI by `tests/test_guide.py`.

```
import warnings

from ikarem import Ikarem, deprecated
from ikarem.testing import TestClient

@deprecated("use total_v2() instead", since="1.1.0", removal="2.0.0", use_instead="total_v2")
def total(items):
    return sum(items)

def total_v2(items):
    return sum(items)

app = Ikarem(enable_docs=False)

@app.get("/total")
async def total_h(req):
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        n = total([1, 2, 3])
    assert any("1.1.0" in str(w.message) and "total_v2" in str(w.message) for w in caught)
    return {"n": n}

assert TestClient(app).get("/total").json() == {"n": 6}
```